

LangChain & LangGraph

Beginner-to-Interview Documentation

A practical guide to understanding what LangChain and LangGraph are, why they are used, how they work, how they differ from normal Python code, and how to build simple workflows.

1. The Big Picture

An LLM application can be as simple as User → LLM → Answer. For real applications, however, we often need prompts, tools, web search, documents, vector databases, retrievers, structured outputs, agents, decisions, loops, and shared state.

Simple application:

User → LLM → Answer

More realistic application:

User → Prompt → LLM → Tool → Data → LLM → Answer

LangChain mainly gives you reusable building blocks for LLM applications. LangGraph mainly gives you a way to orchestrate complex, stateful workflows using those building blocks.

2. What is LangChain?

LangChain is a framework for developing applications powered by large language models (LLMs). It provides integrations and abstractions for models, prompts, tools, document processing, retrieval, structured output, agents, and chains.

The easiest mental model:

LangChain = building blocks for an LLM application

LLM + Prompt + Tool + Retriever + Parser + Agent

↓

Application

3. Is LangChain an LLM?

No.

- OpenAI, Anthropic, Google, etc. provide models.
- LangChain is an application framework that helps you use models and connect them with other components.
- LangChain does not train your LLM.

4. Why use LangChain instead of only normal Python?

You do not have to use LangChain. A small application can be written directly with a model provider's SDK. LangChain becomes useful when you have many reusable AI components and integrations.

Without a framework:

Python → API → LLM

Python → Search API

Python → Vector DB

Python → Parser

You manually connect everything.

With LangChain:

Prompt → Model → Parser

Question → Retriever → Documents → Prompt → Model

Agent → Tools → Model

The important difference is not that LangChain makes Python possible. It is that LangChain gives common interfaces and composable components so larger LLM applications are easier to build and change.

5. Core LangChain Concepts

5.1 Chat Models

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5")

response = model.invoke("Explain machine learning")
print(response.content)
```

invoke() means: execute this component with the supplied input and return its result.

5.2 Prompt Templates

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_template(
    "Explain {topic} to a beginner."
)

prompt_value = prompt.invoke({"topic": "LangChain"})
```

A prompt template lets you create reusable prompts instead of hard-coding a different prompt every time.

5.3 Chains and the | operator

```
chain = prompt | model

response = chain.invoke({
    "topic": "LangChain"
})

print(response.content)
```

The | operator composes components. The output of the component on the left becomes the input of the component on the right.

```
prompt | model
Prompt → Model
```

```
prompt | model | parser  
Prompt → Model → Parser
```

5.4 Tools

A tool is a function an AI system can use to perform an action or obtain information outside the model itself.

```
from langchain_core.tools import tool  
  
@tool  
def add(a: int, b: int) -> int:  
    "Add two numbers."  
    return a + b
```

Typical tools include web search, calculators, databases, APIs, Python functions, and file operations.

5.5 Tool Calling and Agents

An agent can decide which available tool to use based on the user's request and the tool descriptions.

```
User  
↓  
LLM / Agent  
↓  
Choose a tool  
↓  
Tool  
↓  
Tool result  
↓  
LLM / Agent  
↓  
Final answer
```

The key idea is that a normal fixed chain follows a known sequence, while an agent can select actions dynamically.

5.6 Documents, Text Splitters, Embeddings, Vector Stores, and Retrievers

These are common building blocks for RAG (Retrieval-Augmented Generation).

```
Documents  
↓  
Split into chunks  
↓  
Embeddings  
↓  
Vector store  
  
User question  
↓  
Retriever  
↓  
Relevant chunks  
↓  
Prompt + chunks  
↓  
LLM  
↓  
Answer
```

- Document loader: reads data from PDFs, web pages, text files, etc.
- Text splitter: breaks large documents into manageable chunks.
- Embedding: converts text into numerical vectors representing semantic meaning.
- Vector store: stores embeddings for similarity search.
- Retriever: finds relevant documents/chunks for a query.

6. A Small LangChain Example

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

prompt = ChatPromptTemplate.from_template(
    "Explain {topic} in 5 simple points."
)

model = ChatOpenAI(model="gpt-5")

chain = prompt | model

result = chain.invoke({"topic": "RAG"})

print(result.content)
```

Mental execution:

```
{"topic": "RAG"}
  ↓
Prompt template
  ↓
"Explain RAG in 5 simple points."
  ↓
Chat model
  ↓
Response
```

7. What is LangGraph?

LangGraph is a framework for building stateful, multi-step, agentic workflows as graphs. It is especially useful when an application needs branching, loops, retries, shared state, multi-agent coordination, persistence, or human-in-the-loop steps.

LangGraph = workflow orchestration

```
State
  ↓
Node → Node → Node
      ↓
    Decision
    ↙   ↘
  Loop   End
```

8. The Three Core LangGraph Concepts

8.1 State

State is the shared information carried through the workflow. Think of it as a shared notebook.

```
from typing import TypedDict

class State(TypedDict):
    question: str
    search_results: list
    documents: list
    answer: str
```

A node reads the current state and returns updates to it.

8.2 Nodes

A node is usually a Python function that performs one step of the workflow.

```
def search_node(state: State):
    results = web_search(state["question"])
    return {"search_results": results}
```

8.3 Edges

An edge controls which node runs next.

```
graph.add_edge(START, "search")
graph.add_edge("search", "reader")
graph.add_edge("reader", "writer")
graph.add_edge("writer", END)
```

9. Basic LangGraph Code

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    question: str
    search_results: list
    documents: list
    answer: str

def search_node(state):
    results = web_search(state["question"])
    return {"search_results": results}

def reader_node(state):
    docs = read_pages(state["search_results"])
    return {"documents": docs}

def writer_node(state):
    answer = generate_answer(state["documents"])
    return {"answer": answer}

graph = StateGraph(State)
```

```

graph.add_node("search", search_node)
graph.add_node("reader", reader_node)
graph.add_node("writer", writer_node)

graph.add_edge(START, "search")
graph.add_edge("search", "reader")
graph.add_edge("reader", "writer")
graph.add_edge("writer", END)

app = graph.compile()

result = app.invoke({
    "question": "What is RAG?",
    "search_results": [],
    "documents": [],
    "answer": ""
})

print(result["answer"])

```

10. Conditional Edges and Loops

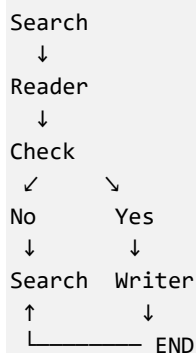
This is one of the biggest reasons to use LangGraph. The workflow can inspect state and choose a path.

```

def check_information(state):
    if len(state["documents"]) >= 3:
        return "writer"
    return "search"

graph.add_conditional_edges(
    "reader",
    check_information,
    {
        "writer": "writer",
        "search": "search"
    }
)

```



Normal Python can also implement this loop. LangGraph's advantage is that the workflow is represented explicitly as nodes, transitions, and state, which makes complex agent workflows easier to reason about.

11. LangChain vs LangGraph

Question	LangChain	LangGraph
Main purpose	LLM application building blocks	Workflow orchestration
Main ideas	Models, prompts, tools, retrievers, parsers, agents	State, nodes, edges, routing, loops
Linear pipeline	Very suitable	Can represent it, but not always necessary
Complex branching	Possible, but not its core focus	Core use case
Loops / retries	Can be implemented	Natural graph pattern
Multi-agent workflow	Agent components available	Strong orchestration fit
Human-in-the-loop	Possible	Strong workflow fit
Shared workflow state	Use-case dependent	Core concept

12. The Best Mental Model

LangChain:

"What components do I need?"

LLM

Prompt

Tool

Retriever

Parser

Agent

LangGraph:

"How should those components work together?"

START

↓

Node

↓

Node

↓

Decision

↙ ↘

Loop Next node

↓

END

13. LangChain + LangGraph Together

They are complementary. A LangGraph node can use LangChain models, prompts, tools, retrievers, and other components.

LANGGRAPH

↓

Search Node

↓

LangChain

Web Search Tool

↓

```

graph TD
    A[Search Results] --> B[Reader Node]
    B --> C[LangChain LLM]
    C --> D[Writer Node]
    D --> E[Answer]

```

14. Example: Multi-Agent Research Assistant

A research assistant can have specialized agents or nodes for searching, reading, writing, and checking.

```

graph TD
    User --> Planner
    Planner --> Search_Agent[Search Agent]
    Search_Agent --> Reader_Agent[Reader Agent]
    Reader_Agent --> Writer_Agent[Writer Agent]
    Writer_Agent --> Critic
    Critic --> Check{Is answer good?}
    Check -- No --> Writer
    Check -- Yes --> END

```

Possible state:

```

class State(TypedDict):
    question: str
    search_results: list
    documents: list
    draft: str
    feedback: str
    final_answer: str

```

15. When Should You Use Which?

Use LangChain when:

- You need reusable LLM application components.
- You need prompt templates, model interfaces, tools, retrieval, or structured output.
- You are building a straightforward chain or an agent with tools.

Use LangGraph when:

- Your workflow has multiple steps and shared state.

- You need conditional routing or loops.
- You have multiple agents that need to coordinate.
- You need retries, human approval, persistence, or long-running workflows.

Use neither when:

- A direct model SDK call is all you need.
- Adding a framework would create more complexity than value.

16. Common Interview Questions

What is LangChain?

A framework for building LLM-powered applications using reusable abstractions for models, prompts, tools, retrieval, structured output, agents, and related components.

What is LangGraph?

A framework for building stateful, multi-step, graph-based AI workflows using nodes, edges, and shared state.

Is LangChain an LLM?

No. It is an application framework around LLMs.

Does LangChain train models?

No. It is primarily used to build applications that use existing models.

Why use LangGraph if Python can make workflows?

Python can implement the same logic, but LangGraph makes complex stateful workflows explicit and easier to manage, extend, persist, and debug.

What is a node?

A function or workflow step that reads state and returns state updates.

What is an edge?

A transition that determines which node runs next.

What is state?

Shared information carried through the graph and used by nodes.

Chain vs agent?

A chain follows a predefined sequence; an agent can dynamically decide which actions/tools to take.

17. Learning Roadmap

1. LLM basics: messages, prompts, tokens, context, model APIs.

2. LangChain models and invoke().
3. Prompt templates.
4. Chains and the | operator.
5. Structured output and parsers.
6. Tools and tool calling.
7. Agents.
8. RAG: loaders, chunking, embeddings, vector stores, retrievers.
9. LangGraph: StateGraph, state, nodes, START, END, edges.
10. Conditional edges and loops.
11. Multi-agent workflows, persistence, and human-in-the-loop.

18. Final Cheat Sheet

LLM
= the model that generates/understands text

LangChain
= reusable building blocks for LLM applications

Prompt
= instructions/input given to the model

Chain
= connected sequence of components

Tool
= function/API the AI can use

Agent
= system where the model can choose actions/tools

RAG
= retrieve relevant information and give it to the model

LangGraph
= graph-based orchestration for stateful AI workflows

State
= shared information in a LangGraph workflow

Node
= a workflow step/function

Edge
= connection / transition between nodes

19. One-Sentence Summary

LangChain helps you build the pieces of an LLM application; LangGraph helps you control how those pieces and agents work together in a stateful, branching, looping workflow.

End of documentation